Group Project: Live Bilingual Classroom Translation Application

Group Members: Vinh, Shiming, Yuhe

This project delivers a real-time, privacy-preserving, and completely offline bilingual classroom transcription system. To operate strictly within the End-to-End Delay threshold of <= 3.0 seconds without requiring dedicated GPU acceleration, the software employs a decoupled, multi-threaded pipeline executing entirely on CPU hardware.

Pipeline Component Design & Technical Justification:

1. Microphone - PyAudio
    - uses PyAudio to dynamically search for and connect to the default input audio device
    - It is equipped with automatic downsampling (44.1/48khz to 16khz) and multi-channel compatibility to ensure stable acquisition on different microphones.
2. Optional noise reduction - webrtcvad (STT (audio_energy < 0.01))
    - local WebRTC VAD and alongside an RMS audio energy threshold check
    - Testing accuracy in a noisy environment
3. Live speech-to-text - faster-whisper (base.en model)
    - Based on full CPU, no need for GPU requirement
    - Completely free; no API usage fees
    - Very fast speed with low inference timer per segment
    - High privacy; audio and transcribed text never leave the local machine
    - Can work offline, zero internet dependence
4. Sentence or phrase correction - Ollama (qwen2.5:0.5b)
    - Ollama service running a lightweight local LLM using a structured JSON prompt to correct STT mishearings
    - Lightweight LLM provides extremely low latency
    - Avoiding cloud network latency, costs, and data leaks
    - if Ollama crashes or times out, it automatically switches to built-in regular expression replacement regularization.
5. Translation - Helsinki-NLP/opus-mt-en-zh

- It uses the transformers library (MarianTokenizer and MarianMTModel executed on CPU)
- It runs independently from both the Whisper transcription model and the Ollama correction model

6. Bilingual display - Tkinter/queue
    - Tkinter is built into standard Python, requiring zero third-party GUI framework dependencies
    - The non-blocking thread and queue structure prevent UI freezes during model processing
    - Compared to cross-platform options like PyQt/PySide or Electron, Tkinter is significantly lighter, easier to prototype, and has a much smaller memory footprint
7. End-to-end Delay <= 3s - Whisper base + Ollama 0.5B + MarianMT around 1-3 sec.
    - The complete local pipeline achieves an end-to-end processing time of 1.0 to 2.2 seconds on a standard CPU—well below the 3.0-second threshold. By combining faster-whisper (int8 quantized STT, ~0.8s), Ollama qwen2.5:0.5b (lightweight LLM correction, ~0.3s), and MarianMT (specialized translation, ~0.2s), the architecture completely eliminates cloud network latency while ensuring immediate incremental UI updates.

Comparative Analysis of Rejected Alternatives:

1. Transcription: why not choose cloud apis (such as openai whisper api) or vosk/coqui?
    - cloud apis offer high accuracy but come with network latency, pay-as-you-go costs, and data privacy risks.
    - The native whispering implementation relies too heavily on pytorch and has slow cpu inference.
2. Correction: why not choose cloud LLM (gpt-4o-mini) or a larger local LLM (qwen-7b)?
    - cloud LLM adds 1-2 seconds of api network round-trip latency; the 7b local model experiences a first token latency exceeding 3-5 seconds on the cpu, directly causing a total latency timeout (3 seconds)

3. Translation: why not just let qwen-0.5b do both "error correction and translation" in one go?
    - small models at the 0.5b level are prone to output format corruption and hallucinations when handling complex instructions simultaneously (both error correction and translation)
    - splitting the output into Ollama (pure error correction) + MarianMT (specialized translation) ensures extremely stable output.

The Demonstration example:



